

FORTRAN JSON RPC 2.0 STATIC LIBRARY DEMONSTRATION PROGRAMS USER MANUAL

REVISION 0

06/21/26

**FORTTRAN JSON RPC 2.0 STATIC LIBRARY DEMONSTRATION PROGRAMS USER MANUAL
REV. 0, 06/21/26**

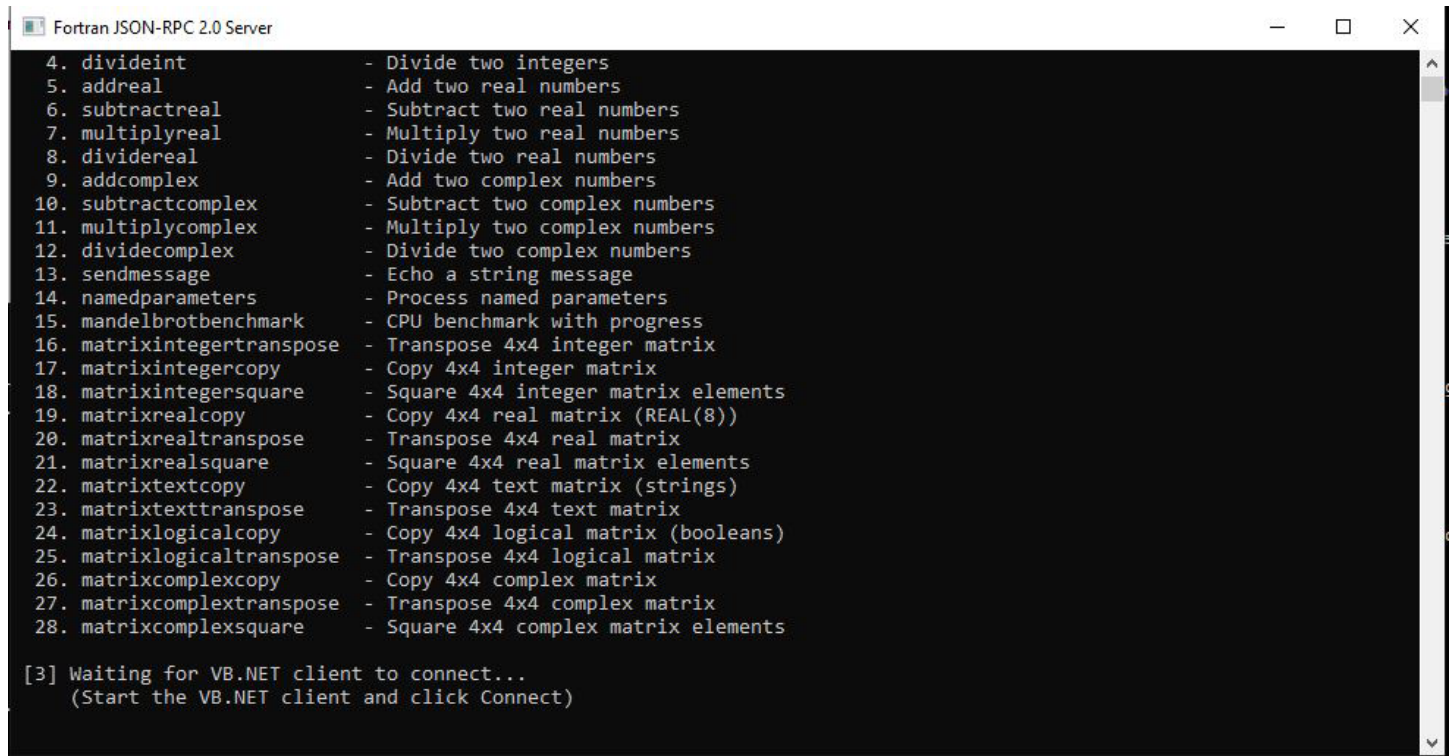
The following provides guidance on using the VB.NET client program in conjunction with the Fortran server program, and provides insight as to the various functions implemented. In general, for each major feature of the demonstration program, graphics of the feature, along with notes are provided. Note that the VB.NET client program and Fortran server program both have text based 'consoles', which help in understanding what the programs are doing. The text from each console for each major feature is provided as a reference, and to demonstrate the expected content of each server.

Notes:

1. If you have not already done so, be sure to compile the static library and the Fortran server program. Same for the VB client program and DLL.
2. Note that the server is a Fortran program (executable), and not just the static library. The VB client communicates with the Fortran server program; the Fortran server program uses the JSON RPC static library.
3. Both programs provide information on their respective consoles such that the user can see what information is being exchanged during the inter-process communication between the client and the server. This level of information is not needed for a production implementation, but is provided for diagnostics and understanding of the processes.
4. The client program is halted by pressing the "Close" button in the bottom right corner of Form1. If you are not familiar with Fortran console programs, you will need to click on the console window and press Enter when prompted, or click on the X in the upper right corner of the console window.
5. These demonstration programs do not provide an extensive or complete example of every possible operation defined by the JSON RPC 2.0 standard, but rather include what could be considered as the most commonly used operations for a given application. It is assumed that a developer/user would expand/modify this code base as needed to implement JSON RPC features not found in this proof of concept series of programs.

Startup

The Fortran server ('server') should be started before launching the VB.NET client ('client') program. The server console should look like this:



```
Fortran JSON-RPC 2.0 Server

4. divideint           - Divide two integers
5. addreal             - Add two real numbers
6. subtractreal        - Subtract two real numbers
7. multiplyreal        - Multiply two real numbers
8. dividereal          - Divide two real numbers
9. addcomplex          - Add two complex numbers
10. subtractcomplex     - Subtract two complex numbers
11. multiplycomplex     - Multiply two complex numbers
12. dividecomplex      - Divide two complex numbers
13. sendmessage        - Echo a string message
14. namedparameters    - Process named parameters
15. mandelbrotbenchmark - CPU benchmark with progress
16. matrixintegertranspose - Transpose 4x4 integer matrix
17. matrixintegercopy   - Copy 4x4 integer matrix
18. matrixintegerstquare - Square 4x4 integer matrix elements
19. matrixrealcopy      - Copy 4x4 real matrix (REAL(8))
20. matrixrealtranspose - Transpose 4x4 real matrix
21. matrixrealstquare   - Square 4x4 real matrix elements
22. matrixtextcopy      - Copy 4x4 text matrix (strings)
23. matrixtexttranspose - Transpose 4x4 text matrix
24. matrixlogicalcopy   - Copy 4x4 logical matrix (booleans)
25. matrixlogicaltranspose - Transpose 4x4 logical matrix
26. matrixcomplexcopy   - Copy 4x4 complex matrix
27. matrixcomplextranspose - Transpose 4x4 complex matrix
28. matrixcomplexsquare - Square 4x4 complex matrix elements

[3] Waiting for VB.NET client to connect...
    (Start the VB.NET client and click Connect)
```

The client program is a single form Windows program, and the form (Form1) is shown below:

Connect to server

Connect **Disconnected** Disconnect

Send message

Send message Message to send Clear message

Send notification

Send notification Notification to send Clear notification

Positional parameters

Select method: Param 1 Param 2 Execute

Named parameters

Enter named parameters: Send named parameters

Test for error codes

Test error 32700
Test error 32600
Test error 32601
Test error 332602
Test error 32603

Test for batch request

Test batch request

Matrix operations

Select data type: Select operation: Select row/column ordering: Execute

Original matrix

(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

Modified matrix

(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

Interactive demo

Enter an integer (1 - 10): Start demo program
Pause demo
Resume demo
Exit demo
Close

VB.NET Client console

```
08:25:35.295 Client program started.
08:25:35.343 Ready to connect to Fortran JSON-RPC server...
08:25:35.367 Matrix labels initialized successfully (lblOM)
08:25:35.389 Matrix labels initialized successfully (lblMM)
```

Clear log

Here is the full text for the server startup console:

```
=====
Fortran JSON-RPC 2.0 Server
=====
```

```
[1] Creating named pipe: MyTestPipe
[SUCCESS] Named pipe created
```

```
[2] JSON-RPC 2.0 Server started
    Pipe name: MyTestPipe
    Mode: Synchronous
    Direction: Duplex (InOut)
    Registered methods: 28
    Method matching: Case-sensitive
    Error Codes: JSON-RPC 2.0 Standard
    Max request size: 1048576 bytes
```

Available Methods:

- | | |
|---------------------|--------------------------------|
| 1. addint | - Add two integers |
| 2. subtractint | - Subtract two integers |
| 3. multiplyint | - Multiply two integers |
| 4. divideint | - Divide two integers |
| 5. addreal | - Add two real numbers |
| 6. subtractreal | - Subtract two real numbers |
| 7. multiplyreal | - Multiply two real numbers |
| 8. dividereal | - Divide two real numbers |
| 9. addcomplex | - Add two complex numbers |
| 10. subtractcomplex | - Subtract two complex numbers |

- | | |
|----------------------------|--------------------------------------|
| 11. multiplycomplex | - Multiply two complex numbers |
| 12. dividecomplex | - Divide two complex numbers |
| 13. sendmessage | - Echo a string message |
| 14. namedparameters | - Process named parameters |
| 15. mandelbrotbenchmark | - CPU benchmark with progress |
| 16. matrixintegertranspose | - Transpose 4x4 integer matrix |
| 17. matrixintegercopy | - Copy 4x4 integer matrix |
| 18. matrixintegerstquare | - Square 4x4 integer matrix elements |
| 19. matrixrealcopy | - Copy 4x4 real matrix (REAL(8)) |
| 20. matrixrealtranspose | - Transpose 4x4 real matrix |
| 21. matrixrealsquare | - Square 4x4 real matrix elements |
| 22. matrixtextcopy | - Copy 4x4 text matrix (strings) |
| 23. matrixtextttranspose | - Transpose 4x4 text matrix |
| 24. matrixlogicalcopy | - Copy 4x4 logical matrix (booleans) |
| 25. matrixlogicaltranspose | - Transpose 4x4 logical matrix |
| 26. matrixcomplexcopy | - Copy 4x4 complex matrix |
| 27. matrixcomplextranspose | - Transpose 4x4 complex matrix |
| 28. matrixcomplexsquare | - Square 4x4 complex matrix elements |

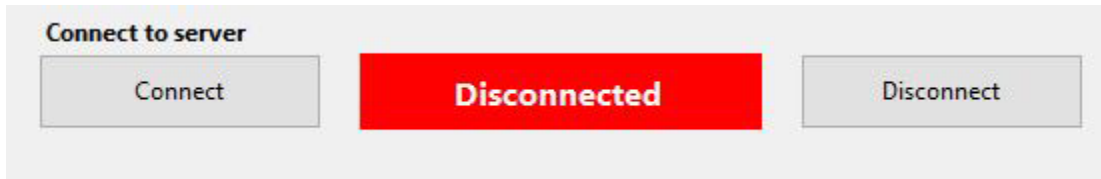
[3] Waiting for VB.NET client to connect...
(Start the VB.NET client and click Connect)

Here is the full text for the client window upon startup (not connected yet)

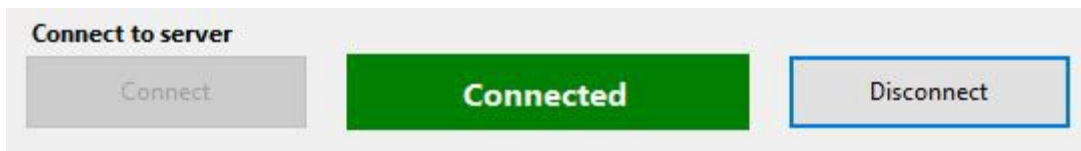
08:36:08.172 Client program started.
08:36:08.209 Ready to connect to Fortran JSON-RPC server...
08:36:08.234 Matrix labels initialized successfully (lblOM)
08:36:08.255 Matrix labels initialized successfully (lblMM)

Connection/disconnect

The client objects associated with the connection of the client to the server are:



Use the "Connect" button to connect to the server program, after successful connection, the "Disconnected" label changes to green and the label says "Connected" as below:



When you are through with the demo program, click on "Disconnect" and then the "Close" button on the bottom right of Form1. The label in the middle will change back to red and say "Disconnected".

Here is the expected text of the client and server consoles after connection is complete:

VB CONSOLE TEXT AFTER CONNECTION

```
08:37:10.517 Connecting to Fortran server...
08:37:10.644 Connected to named pipe.
08:37:10.645 JSON-RPC initialized with Content-Length protocol.
08:37:10.646 Full duplex communication ready.
```

FORTTRAN CONSOLE TEXT AFTER CONNECTION

```
=====
[SUCCESS] VB.NET Client Connected!
=====
```

```
Server is now ready to:
- Receive JSON-RPC 2.0 requests
- Process Batch requests
- Send real-time progress notifications
- Process AddInt calls
- Process SubtractInt calls
- Process MultiplyInt calls
- Process AddReal calls
- Process SubtractReal calls
- Process MultiplyReal calls
- Process DivideReal calls (with div-by-zero check)
- Process DivideInt calls (with div-by-zero check)
- Process AddComplex calls
- Process SubtractComplex calls
- Process MultiplyComplex calls
- Process DivideComplex calls (with div-by-zero check)
- Process NamedParameters calls
- Process MandelbrotBenchmark calls
```

- Process MatrixIntegerTranspose calls
- Process MatrixIntegerCopy calls
- Process MatrixIntegerSquare calls
- Process MatrixRealCopy calls
- Process MatrixRealTranspose calls
- Process MatrixRealSquare calls
- Process MatrixTextCopy calls
- Process MatrixTextTranspose calls
- Process MatrixLogicalCopy calls
- Process MatrixLogicalTranspose calls
- Process MatrixComplexCopy calls
- Process MatrixComplexTranspose calls
- Process MatrixComplexSquare calls
- Handle Notifications (no response)
- Return standard error codes (-32700 to -32000)

Press Ctrl+C to stop server

=====

Send message

The objects associated with sending a message to the server are:

Enter a message in the “Message to send” textbox, and then click “Send message” button. Clear out the textbox by clicking on the “Clear message button”. After sending a message from the client to the server, you will see text similar to the following:

VB CONSOLE

```
08:39:44.673 =====
08:39:44.674 -> SENDING to server: "this is a message"
08:39:44.676     Method: sendmessage
08:39:44.896 =====
08:39:44.898 <- ECHO RESPONSE from server: "this is a message"
08:39:44.900 =====
08:39:44.901 ECHO VERIFIED - Message matches perfectly!
08:39:44.904     Sent:      "this is a message"
08:39:44.905     Received: "this is a message"
08:39:44.907 =====
08:39:44.908
```

FORTTRAN CONSOLE

```
08:39:44 [RECEIVED] 100 bytes
08:39:44 JSON content: {"jsonrpc":"2.0","id":2,"method":"sendmessage","params":["this is
a message"]}
08:39:44 [TYPE] Request (response expected)

SendMessage: this is a message

08:39:44 JSON response:
{
  "jsonrpc": "2.0",
  "result": "this is a message",
  "id": 2
}

08:39:44 Sending response...

08:39:44 Response sent (      89  bytes)
```


Send notification

This is almost identical to sending a message. The difference is that when a message is sent to the server, the server is obligated to send a response. When a notification is sent, by default, a response from the server is not sent. The objects on Form 1 associated with sending a notification are:

The screenshot shows a web form titled "Send notification". It contains a "Send notification" button on the left, a text input field in the center with the value "this is a notification", and a "Clear notification" button on the right. Below the input field, the text "Positional parameters" is visible.

The expected text in the consoles are as follows (note that the server acknowledges that no response is sent for a notification).

VB CONSOLE

```
08:42:25.400 =====
08:42:25.401 SENDING NOTIFICATION to server: "this is a notification"
08:42:25.402 (No response expected - JSON-RPC 2.0 Notification)
08:42:25.441 Notification dispatched to server
08:42:25.442 Delivery unconfirmed - no acknowledgment per JSON-RPC 2.0 spec
08:42:25.447 =====
08:42:25.448
```

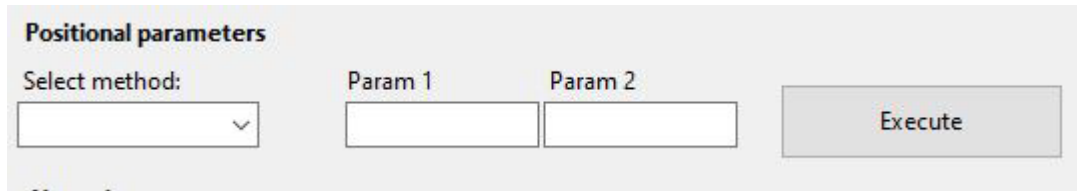
FORTTRAN CONSOLE

```
08:42:25 [RECEIVED] 98 bytes
08:42:25 JSON content: {"jsonrpc":"2.0","method":"clientevent","params":["this is a
notification"]}
08:42:25 [TYPE] Notification (no response expected)

08:42:25 [NOTIFICATION] No response sent (notification processed)
```

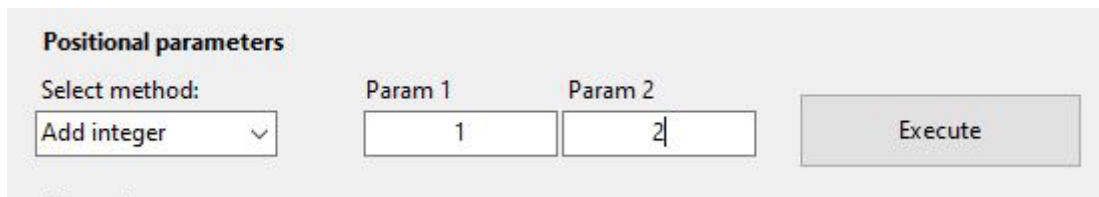
Positional parameters – add integers example

Here we are calling for a ‘method’ (basically a function) on the server to take two input parameters and provide a response. This example adds two integers input by the user, and displays the response from the server. The objects associated with the positional parameters demo are:



The screenshot shows a web interface titled "Positional parameters". It contains a "Select method:" dropdown menu, two text input fields labeled "Param 1" and "Param 2", and a button labeled "Execute".

Note that the “Select method:” combobox allows the selection of four arithmetic functions (add, subtract, multiply and divide) with the variable types integer, real, and complex. For this example, to add integers, select “Add integer” from the dropdown combobox and enter an integer into the Param 1 textbox, and another in the Param 2 textbox, then click on the “Execute” button. Also note that the program validates the type of data input into the Param 1 and Param 2 textboxes to insure that the data matches the selected data type for the variables. Here is the same objects with the input data (two integers) and the Add integer method selected:



This screenshot shows the same interface as the previous one, but with the "Select method:" dropdown set to "Add integer", "Param 1" containing the value "1", and "Param 2" containing the value "2".

The method on the server associated with this is “addinteger”, and it returns the sum of the two numbers which is displayed in a messagebox when complete.

The expected text for the client and server consoles is:

VB CONSOLE INTEGERS

```
08:45:28.623 =====
08:45:28.625 CALLING addint on Fortran server
08:45:28.627     Operation: Addition
08:45:28.630     Parameter 1: 1
08:45:28.631     Parameter 2: 2
08:45:28.632     Expression: 1 + 2
08:45:28.650 =====
08:45:28.653 <- RESULT from Fortran server: 3
08:45:28.654 =====
08:45:28.656 Addition verified: 1 + 2 = 3
08:45:30.116 =====
08:45:30.118
```

FORTRAN CONSOLE INTEGERS

```
08:45:28 [RECEIVED] 79 bytes
08:45:28 JSON content: {"jsonrpc":"2.0","id":3,"method":"addint","params":[1,2]}
08:45:28 [TYPE] Request (response expected)
```

AddInt: 1 + 2 = 3

```
08:45:28 JSON response: {  
  "jsonrpc": "2.0",  
  "result": 3,  
  "id": 3  
}
```

08:45:28 Sending response...

08:45:28 Response sent (71 bytes)

Positional parameters – add complex numbers example

This is identical to the previous example with integers, except that the user is inputting two complex numbers. The basic format for a complex number is

$(R + li)$.

Where R is the “real” number and “l” is the imaginary number (followed by a lower case “l”). Both the real and imaginary values can be integer or real values. Real values can be of the format -####.## to ####.## (for simplicity, this is not a constraint of VB, JSON or Fortran, just a convenience). Scientific notation can be of the format -####.##E## to ####.##E##; note that the two digit exponent can be either positive or negative. The input format by the user must comply with the following representation for a complex number:

$(\# + \#i)$

Which is to say left parenthesis, number, space, + or -, space number, l, right parenthesis

Examples include:

(5 + 10i)
 (-5 - 10i)
 (5.50 + 11.11i)
 (1234.56 - 7890.11E43)
 (-1234.56 + 7890.11E-43)

As before, the objects on Form1 for this operation are:

The method on the server associated with this is “addcomplex”, and it returns the sum of the two numbers which is displayed in a messagebox when complete.

The expected text for the client and server consoles is:

VB CONSOLE

```
08:48:42.951 =====
08:48:42.953 CALLING addcomplex on Fortran server
08:48:42.955     Operation : Complex Addition
08:48:42.956     Parameter 1: 10 + 23i
08:48:42.957     Parameter 2: 43 + 8i
08:48:42.973 =====
08:48:42.975 <- RAW RESULT from Fortran server: 53+31i
08:48:42.979 <- FORMATTED RESULT: 53 + 31i
08:48:42.981 =====
08:48:42.984 Verification OK: (10 + 23i) + (43 + 8i) = 53 + 31i
08:48:44.299 =====
08:48:44.301
```

FORTRAN CONSOLE

```

08:48:42 [RECEIVED] 100 bytes
08:48:42 JSON content: {"jsonrpc":"2.0","id":4,"method":"addcomplex","params":["10 +
23i","43 + 8i"]}
08:48:42 [TYPE] Request (response expected)

AddComplex: ( 10.000000000000000 + 23.000000000000000 i) + (
43.000000000000000 + 8.000000000000000 i) = (
53.000000000000000 + 31.000000000000000 i)

08:48:42 JSON response:
{
  "jsonrpc": "2.0",
  "result": "53+31i",
  "id": 4
}

08:48:42 Sending response...

08:48:42 Response sent ( 78 bytes)

```

Name parameters

Named parameters in JSON-RPC refer to a method of passing arguments as a JSON Object, where each argument is paired with its corresponding, predefined parameter name rather than relying on its specific index or position in an array. In this example, we are transferring the following variables and their numeric values:

Variable	Value
X	43
Y	56
P	123.45
Num	0.89

So this is a nice alternative to send multiple data but not as an array.

The objects on Form1 associated with this operation are:

The user should enter one or more named parameters, of the general format:

“variable1”: value1, “variable2”: value2,.....

For this example, the user input is:

Clicking on the “Send named parameters” will transfer the input to the server, which provides a response, which the client will display in a messagebox. For demonstration purposes, the server adds up the number of individual variables and the response is the sum. Once again, this technique is an interesting alternative to send multiple data values in lieu of the use of arrays or matrices.

VB CONSOLE

```

08:51:30.874  =====
08:51:30.875  -> SENDING NAMED PARAMETERS to server
08:51:30.876      Method: namedparameters
08:51:30.877      Parameters: {"x":43, "y":56, "p":123.45, "num":0.89}
08:51:30.907  =====
08:51:30.908  <- RESPONSE from server:
08:51:30.909      The total number of named parameters received is: 4
08:51:30.910  =====
08:51:30.912
    
```

FORTTRAN CONSOLE

```

08:51:30 [RECEIVED] 120 bytes
    
```

08:51:30 JSON content:

```
{"jsonrpc": "2.0", "id": 5, "method": "namedparameters", "params": {"x": 43, "y": 56, "p": 123.45, "num": 0.89}}
```

08:51:30 [TYPE] Request (response expected)

NamedParameters: Received 4 named parameter(s):

KEY: x, VALUE: 43, TYPE: INTEGER

KEY: y, VALUE: 56, TYPE: INTEGER

KEY: p, VALUE: 0.123450000000000000E+0003, TYPE: REAL

KEY: num, VALUE: 0.890000000000000001E+0000, TYPE: REAL

Total named parameters: 4

08:51:30 JSON response:

```
{  
  "jsonrpc": "2.0",  
  "result": "The total number of named parameters received is: 4",  
  "id": 5  
}
```

08:51:30 Sending response...

08:51:30 Response sent (124 bytes)

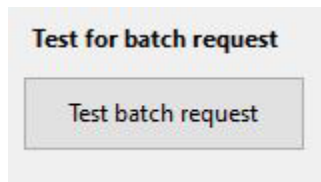
Test batch request

JSON RPC allows the user to send multiple messages as one “batch”. Unfortunately, as noted in the other documentation, the Microsoft library StreamJsonRpc, which is used by .NET programs to implement the JSON RPC protocol, does not allow for fully compliant (with the JSON RPC 2.0 standard) formation of the batch request. Regardless, the batch request does “work”, just not the way the standard would require.

There is no user input. The batch request built into the client is:

```
subtractint(100, 25)
multiplyint(6, 7)
addint(100, 456)
Notification: 'Batch test notification'
```

The input for the batch request is just the one button:



Here is the expected text for the batch request:

VB CONSOLE

```
08:54:24.378 =====
08:54:24.379 TEST: Batch Request
08:54:24.380     Sending multiple concurrent requests to server...
08:54:24.381     - subtractint(100, 25)
08:54:24.381     - multiplyint(6, 7)
08:54:24.382     - addint(100, 456)
08:54:24.383     - Notification: 'Batch test notification'
08:54:24.405 =====
08:54:24.408 BATCH REQUEST RESULTS:
08:54:24.411     subtractint(100, 25) = 75   (expected 75)
08:54:24.413     multiplyint(6, 7)   = 42   (expected 42)
08:54:24.415     addint(100, 456)   = 556   (expected 556)
08:54:24.417     Calculation verified: 100 + 456 = 556
08:54:24.418     Notification sent: True
08:54:24.419 =====
08:54:24.420 ✓ All batch results verified successfully!
08:54:26.161
```

FORTRAN CONSOLE

```
08:54:24 [RECEIVED] 99 bytes
08:54:24 JSON content: {"jsonrpc":"2.0","method":"clientevent","params":["Batch test notification"]}
08:54:24 [TYPE] Notification (no response expected)

08:54:24 [NOTIFICATION] No response sent (notification processed)
```


Matrix operations - integers

Fortran loves matrices, and the demo program provides a wide variety of variations on a theme. This part of the demo allows for the generation of random values to fill a 4 x 4 matrix, based upon the following input from the user:

Data type: integer, real, complex, logical/Boolean and character/text

Operation: copy, transpose, square (not used with logical/Boolean or character/text)

Ordering: row-major or column-major

Note that the matrix ordering is for “information only” to assist (if needed) in addressing the matrix ordering for Fortran (column-major) versus other languages (typically row-major). For demo purpose, do not select the column-major option (for simplicity).

Below is the objects on Form1 associated with the matrix operations:

Matrix operations

Select data type:

Select operation:

Select row/column ordering:

Original matrix

(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

Execute

Modified matrix

(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

The sequence is select data type, operation and row-major, and click on the “Execute” button. The original matrix is randomly generated, and sent to the server. The server will execute the appropriate matrix method and returns the modified matrix which is displayed in the second matrix. The resulting console text is provided below. If you review the console text, you will see that the 4 x4 matrices are “flattened” into a 1D array when sent back and forth.

VB CONSOLE

```

08:56:57.182 Generating random integer matrix (0-100)...
08:56:57.309 Random integer matrix generated successfully
08:56:57.324 Available operations updated for 'integer': transpose, copy, square
08:57:09.159 =====
08:57:09.160 MATRIX OPERATION REQUESTED
08:57:09.161 ✓ Connection validated
08:57:09.167 Data Type: integer
    
```

```

08:57:09.169      Operation: transpose
08:57:09.171      Ordering: row-major
08:57:09.173  ✓ Selections validated
08:57:09.176  -----
08:57:09.177  EXECUTING: Integer Matrix Transpose
08:57:09.179  ✓ Matrix is not empty
08:57:09.181  ✓ Matrix dimensions validated: 4x4
08:57:09.188  ✓ All cells contain valid integer data
08:57:09.191  ✓ Matrix extracted: 16 values
08:57:09.197      Matrix values: [70, 59, 45, 23, 14, 5, 91, 48...]
08:57:09.201      Sending to Fortran server...
08:57:09.203      Method: matrixintegertranspose
08:57:09.206      Parameters: datatype=integer, rows=4, columns=4, operation=transpose,
ordering=row-major
08:57:09.256  ✓ Response received from server
08:57:09.259  ✓ Response validated successfully
08:57:09.261      Result matrix: [70, 14, 54, 86, 59, 5, 37, 5...]
08:57:09.332  Matrix displayed: 4x4 = 16 integer values
08:57:09.334  ✓ Result displayed in modified matrix (lblMM)
08:57:09.336  -----
08:57:09.339  INTEGER MATRIX TRANSPOSE COMPLETED SUCCESSFULLY
08:57:09.340  =====
08:57:10.873

```

FORTRAN CONSOLE

```

08:57:09 [RECEIVED] 238 bytes
08:57:09 JSON content:
{"jsonrpc":"2.0","id":9,"method":"matrixintegertranspose","params":{"datatype":"integer",
"rows":4,"columns":4,"operation":"transpose","ordering":"row-
major","matrix":[70,59,45,23,14,5,91,48,54,37,70,93,86,5,64,18]}}
08:57:09 [TYPE] Request (response expected)

```

MatrixIntegerTranspose: 4x4 matrix transposed successfully

08:57:09 JSON response:

```

{
  "jsonrpc": "2.0",
  "result": {
    "datatype": "integer",
    "rows": 4,

    "columns": 4,
    "ordering": "row-major",
    "matrix": [
      70,
      1
4,
      54,
      86,
      59,
      5,
      37,
      5,
      45,

```

```
    91
  ,
    70,
    64,
    23,
    48,
    93,
    18
  ]
},
"id":
9
}
```

08:57:09 Sending response...

08:57:09 Response sent (344 bytes)

Matrix operations – complex numbers

This example is very similar to the previous integer example for matrices, except that complex numbers are the data type, and the operation is to square each element. The resulting objects on Form1 are:

Matrix operations

Select data type: complex
 Select operation: square
 Select row/column ordering: row-major

Execute

Original matrix

(36.13+86.43i)	(20.42+0.35i)	(15.51+85.82i)	(41.01+21.36i)
(44.22+98.67i)	(49.76+10.64i)	(17.86+68.72i)	(91.66+7.39i)
(22.24+67.59i)	(43.16+34.51i)	(9.15+93.14i)	(38.52+15.09i)
(46.24+24.15i)	(18.65+17.61i)	(83.79+89.43i)	(42.22+32.31i)

Modified matrix

(-6164.77+6245.4	(416.85+14.29i)	(-7124.51+2662.1	(1225.57+1751.95i
(-7780.36+8726.3	(2362.85+1058.89	(-4403.46+2454.6	(8346.94+1354.73i
(-4073.79+3006.4	(671.85+2978.90i)	(-8591.34+1704.4	(1256.08+1162.53i
(1554.92+2233.39	(37.71+656.85i)	(-976.96+14986.6	(738.59+2728.26i)

And the resulting console text is:

VB CONSOLE

```

08:59:34.428 Generating random complex matrix (0.00-100.99 + 0.00-100.99i)...
08:59:34.603 Random complex matrix generated successfully
08:59:37.961 =====
08:59:37.962 MATRIX OPERATION REQUESTED
08:59:37.963 ✓ Connection validated
08:59:37.964     Data Type: complex
08:59:37.965     Operation: square
08:59:37.967     Ordering: row-major
08:59:37.969 ✓ Selections validated
08:59:37.972 -----
08:59:37.973 EXECUTING: Complex Matrix Square
08:59:37.975 ✓ Matrix is not empty
08:59:37.977 ✓ Matrix dimensions validated: 4x4
08:59:37.980 ✓ All cells contain valid complex data
08:59:37.982 ✓ Matrix extracted: 16 complex values
08:59:37.986     First 4 values: [(36.13+86.43i), (20.42+0.35i), (15.51+85.82i),
08:59:37.988     (41.01+21.36i)...]
08:59:37.988     Sending to Fortran server...
08:59:37.990     Method: matrixcomplexsquare
08:59:37.992     Parameters: datatype=complex, rows=4, columns=4, operation=square,
ordering=row-major
    
```

```

08:59:38.013  ✓ Response received from server
08:59:38.019  ✓ Response validated successfully
08:59:38.021      First 4 result values: [(-6164.77+6245.43i), (416.85+14.29i), (-
7124.51+2662.14i), (1225.57+1751.95i)]
08:59:38.087  Complex matrix displayed: 4x4 = 16 complex values
08:59:38.089  ✓ Result displayed in modified matrix (lblMM)
08:59:38.091  -----
08:59:38.093  COMPLEX MATRIX SQUARE COMPLETED SUCCESSFULLY
08:59:38.096  =====
08:59:39.713

```

FORTRAN CONSOLE

```

8:59:37 [RECEIVED] 456 bytes
08:59:37 JSON content:
{"jsonrpc":"2.0","id":10,"method":"matrixcomplexsquare","params":{"datatype":"complex","r
ows":4,"columns":4,"operation":"square","ordering":"row-
major","matrix":["(36.13+86.43i)","(20.42+0.35i)","(15.51+85.82i)","(41.01+21.36i)","(44.
22+98.67i)","(49.76+10.64i)","(17.86+68.72i)","(91.66+7.39i)","(22.24+67.59i)","(43.16+34
.51i)","(9.15+93.14i)","(38.52+15.09i)","(46.24+24.15i)","(18.65+17.61i)","(83.79+89.43i)
","(42.22+32.31i)"]}}
08:59:37 [TYPE] Request (response expected)

```

MatrixComplexSquare: 4x4 complex matrix squared successfully

```

08:59:38 JSON response:
{
  "jsonrpc": "2.0",
  "result": {
    "datatype": "complex",
    "rows": 4,

    "columns": 4,
    "ordering": "row-major",
    "matrix": [
      "(-6164.77+
6245.43i)",
      "(416.85+14.29i)",
      "(-7124.51+2662.14i)",
      "(1225.
57+1751.95i)",
      "(-7780.36+8726.37i)",
      "(2362.85+1058.89i)",
      "
(-4403.46+2454.68i)",
      "(8346.94+1354.73i)",
      "(-4073.79+3006.40i)",

      "(671.85+2978.90i)",
      "(-8591.34+1704.46i)",
      "(1256.08+1162.53
i)",
      "(1554.92+2233.39i)",
      "(37.71+656.85i)",
      "(-976.96+14986
.68i)",
      "(738.59+2728.26i)"

```

```
    ]  
  },  
  "id": 10  
}
```

08:59:38 Sending response...

08:59:38 Response sent (634 bytes)

Notifications from a method in the server to the client

One of the advantages of using the JSON RPC protocol with Windows named pipes is that the Fortran server is a standalone executable, and not a library of functions. This allows the server program to do some interesting features that would be hard (or impossible) to do with a function in a library. For this portion of the demo, a method on the server executes a Mandelbrot benchmark program, which is highly computation-intensive, and takes a significant amount of time to complete, even on a modern Windows PC. The user inputs a 'seed' value (integer from 1 to 10) and that is sent to the mandelbrotbenchmark method on the server. Just as a reference, a seed value of 1 can take up to a minute or more to complete on a PC. This 'demo within a demo' provides examples of the following functionalities:

1. Ability of the server to send status notifications back to the client
2. Ability to pause, then resume an operating method on the server
3. Ability to exit a running method on the server

It should be noted that the method noted above is basically a big DO LOOP. The method uses counters to determine when % complete notifications, or check on PAUSE, RESUME and EXIT notifications are received from the client; therefore, these features may not be easy (or possible) to implement on a method that does not involve DO LOOPS.

While it is running, it would be nice to be able to track its progress; this portion of the demo program has the method on the server send notifications to the client program at nominal 10% completion intervals. This is a useful feature in that the console on the VB client Form1 updates with the % complete automatically (as does the Fortran server console).

The objects associated with running the demo program (within the client demo program) are:

The operation is simple in that the user inputs an integer from 1 to 10, and hits the "Start demo program" button. When complete, the resulting text on the client and server consoles is:

VB CONSOLE

```
09:02:58.559 =====
09:02:58.560 CALLING mandelbrotbenchmark on Fortran server
09:02:58.561     Seed: 1
09:02:58.562 Listening for progress notifications...
09:02:58.644 → Progress: 10% complete
09:02:58.966 → Progress: 20% complete
09:03:01.075 → Progress: 30% complete
09:03:05.760 → Progress: 40% complete
09:03:12.843 → Progress: 50% complete
09:03:19.894 → Progress: 60% complete
09:03:24.486 → Progress: 70% complete
```

```
09:03:26.562 → Progress: 80% complete
09:03:26.892 → Progress: 90% complete
09:03:26.941 =====
09:03:26.943 <- CHECKSUM from Fortran server: 115,004,348.01
09:03:26.944 =====
09:03:29.417
```

FORTRAN CONSOLE

```
09:02:58 [RECEIVED] 93 bytes
09:02:58 JSON content:
{"jsonrpc":"2.0","id":11,"method":"mandelbrotbenchmark","params":[1.0]}
09:02:58 [TYPE] Request (response expected)
```

```
=====
MandelbrotBenchmark: Starting computation
=====
Seed value:          1
Grid resolution:      2000 x          2000
Max iterations:       5000
Total points:         4000000
Mode: REQUEST-RESPONSE (with progress notifications)
=====
```

```
10% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[10]}
20% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[20]}
30% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[30]}
40% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[40]}
50% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[50]}
60% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[60]}
70% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[70]}
80% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[80]}
90% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[90]}
```

```
=====
MandelbrotBenchmark: Computation complete
=====
Checksum:    115004348.009204
Format: Real(8) / Double precision
=====
```

Response prepared: returning checksum as result

```
09:03:26 JSON response:
{
  "jsonrpc": "2.0",
  "result": 0.11500434800920366E+9,
  "id": 11
}
```



```
}
```

```
09:03:26 Sending response...
```

```
09:03:26 Response sent (          93  bytes)
```

Pause/resume a method on the server

In this scenario, the user want to pause execution of a running method. After providing a seed integer and pressing Execute, while the method is running, the “Pause” button is active. When this button is clicked, a notification is sent to the server, and the method enters a “holding” loop, and awaits receipt of a “Resume” notification from the client. This can be done multiple times before the method completes, if desired.

The objects on Form1 below show that the user has entered a value of “1”, and the method is running. With the method running, the user may either Pause or Exit the demo (next section).

The screenshot shows a Windows form titled "Interactive demo". On the left, there is a label "Enter an integer (1 - 10):" followed by a text box containing the number "1". To the right of the text box are four buttons stacked vertically: "Start demo program", "Pause demo", "Resume demo", and "Exit demo". The "Pause demo" button is highlighted, indicating it is the active or selected button.

The console text associated with Pausing then Resuming a running method are as follows:

VB CONSOLE

```

09:07:12.719 =====
09:07:12.721 CALLING mandelbrotbenchmark on Fortran server
09:07:12.721     Seed: 1
09:07:12.722 Listening for progress notifications...
09:07:12.778 → Progress: 10% complete
09:07:13.110 → Progress: 20% complete
09:07:15.209 → Progress: 30% complete
09:07:16.048 =====
09:07:16.049 -> Sending pause notification to Fortran server
09:07:16.051     Pause notification sent
09:07:16.053 =====
09:07:16.221 - - - - -
09:07:16.223 ← Server status: "The function has been PAUSED."
09:07:16.225 - - - - -
09:07:16.226
09:07:20.507 =====
09:07:20.508 -> Sending resume notification to Fortran server
09:07:20.510     Resume notification sent
09:07:20.512 =====
09:07:20.556 - - - - -
09:07:20.558 ← Server status: "The function has RESUMED."
09:07:20.559 - - - - -
09:07:20.560
09:07:24.163 → Progress: 40% complete
09:07:31.209 → Progress: 50% complete
09:07:38.264 → Progress: 60% complete
09:07:42.875 → Progress: 70% complete
09:07:44.955 → Progress: 80% complete
09:07:45.283 → Progress: 90% complete

```

```
09:07:45.328 =====
09:07:45.329 <- CHECKSUM from Fortran server: 115,004,348.01
09:07:45.331 =====
09:07:47.580
```

FORTRAN CONSOLE

```
09:07:12 [RECEIVED] 93 bytes
09:07:12 JSON content:
{"jsonrpc":"2.0","id":14,"method":"mandelbrotbenchmark","params":[1.0]}
09:07:12 [TYPE] Request (response expected)

=====
MandelbrotBenchmark: Starting computation
=====
Seed value:          1
Grid resolution:      2000 x          2000
Max iterations:       5000
Total points:         4000000
Mode: REQUEST-RESPONSE (with progress notifications)
=====

10% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[10]}
20% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[20]}
30% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[30]}
check_for_signals: signal=          2 consumed          68 bytes
Benchmark PAUSED by client request
-> Notification:
{"jsonrpc":"2.0","method":"status","params":["The function has been PAUSED."]}
check_for_signals: signal=          3 consumed          69 bytes
Benchmark RESUMED by client request
-> Notification:
{"jsonrpc":"2.0","method":"status","params":["The function has RESUMED."]}
40% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[40]}
50% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[50]}
60% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[60]}
70% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[70]}
80% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[80]}
90% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[90]}

=====
MandelbrotBenchmark: Computation complete
=====
Checksum:      115004348.009204
Format: Real(8) / Double precision
=====

Response prepared: returning checksum as result
```

09:07:45 JSON response:

```
{  
  "jsonrpc": "2.0",  
  "result": 0.11500434800920366E+9,  
  "id": 14  
}
```

09:07:45 Sending response...

09:07:45 Response sent (93 bytes)

Exiting a running method

This is very similar to the Pause/Resume functions, other than the method is directed to exit its main DO LOOP. This does not halt the execution of the server, it merely directs the method DO LOOP to exit the loop and then the server continues normal operation.

The text associated with Exiting a method is:

VB CONSOLE

```
09:09:03.418 =====
09:09:03.419 CALLING mandelbrotbenchmark on Fortran server
09:09:03.420     Seed: 1
09:09:03.420 Listening for progress notifications...
09:09:03.478 → Progress: 10% complete
09:09:03.807 → Progress: 20% complete
09:09:05.889 → Progress: 30% complete
09:09:10.487 → Progress: 40% complete
09:09:17.535 → Progress: 50% complete
09:09:18.671 =====
09:09:18.673 -> Sending stop notification to Fortran server
09:09:18.674     Stop notification sent
09:09:18.676 =====
09:09:19.450 - - - - -
09:09:19.454 ← Server status: "The function has been exited."
09:09:19.457 - - - - -
09:09:19.459
09:09:19.572 =====
09:09:19.574 SERVER ERROR: Benchmark aborted by client request
09:09:19.575     Error code: -32000
09:09:19.580 =====
```

FORTRAN CONSOLE

```
9:09:03 [RECEIVED] 93 bytes
09:09:03 JSON content:
{"jsonrpc":"2.0","id":15,"method":"mandelbrotbenchmark","params":[1.0]}
09:09:03 [TYPE] Request (response expected)

=====
MandelbrotBenchmark: Starting computation
=====
Seed value:          1
Grid resolution:      2000 x 2000
Max iterations:       5000
Total points:         4000000
Mode: REQUEST-RESPONSE (with progress notifications)
=====

10% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[10]}
20% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[20]}
30% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[30]}
```

```

40% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[40]}
50% complete
-> Notification: {"jsonrpc":"2.0","method":"progress","params":[50]}
check_for_signals: signal=          1  consumed          67 bytes
Benchmark stopped by client request
-> Notification:
{"jsonrpc":"2.0","method":"status","params":["The function has been exited."]}
MandelbrotBenchmark: Aborted by client STOP notification

```

```

09:09:19 JSON response:
{
  "jsonrpc": "2.0",
  "error": {
    "code": -32000,
    "message": "Benchmark
k aborted by client request"
  },
  "id": 15
}

```

09:09:19 Sending response...

09:09:19 Response sent (149 bytes)

JSON RPC defined errors

The JSON RPC standard provides for predefined errors. These errors (sent from the server to the client) are as follows:

- **-32700 (Parse error)**: This error triggers when the server receives malformed JSON text that cannot be parsed into a JavaScript object.
- **-32600 (Invalid Request)**: This error triggers when the sent JSON is structured correctly but does not conform to a valid JSON-RPC Request object.
- **-32601 (Method not found)**: This error triggers when the requested method name does not exist or is unavailable on the target server.
- **-32602 (Invalid params)**: This error triggers when the parameters provided for the method are incorrect in number, structured data type, or structural validation.
- **-32603 (Internal error)**: This error triggers when an unexpected primitive execution or system failure occurs entirely on the server side.
- **-32000 to -32099 (Server error)**: This error range triggers when implementation-specific application errors occur that are explicitly reserved for custom server behaviors.

The objects on Form1 associated with testing the ability of the server to recognize these errors and report them are just a series of buttons, with the associated JSON RPC error code on the button:



Console text is not provided with this example. Please note the following:

1. For each button, a predefined message, value, etc. is hardcoded into the client, which sends the text to the server to trigger the individual error message.
2. By design, errors -32700 and -32600 will not only provide an error response, but will shut down the server. Once again, this is in compliance with the JSON RPC 2.0 standard.
3. The lower 3 buttons will generate an informative messagebox that lists the error number and text regarding the cause of the error.
4. A search of the code associated with these 5 buttons will provide the text sent to the server to trigger the individual error.

5. Last but not least, note that the user can arbitrarily assign the error numbers -32000 to -32099 as they see fit for their application.